the same initial state, with a base policy that has identical components, and use the base policy for signaling, the agents will select identical controls under the corresponding multiagent rollout policy, ending up with a potentially serious cost deterioration.

This example also highlights an effect of the sequential choice of the control components $u_k^1, \ldots, u_k^m$, based on the reformulated problem of Fig. 2.9.1: it tends to break symmetries and "group think" that guides the agents towards selecting the same controls under identical conditions. Generally, any sensible multiagent policy must be able to deal in some way with this "group think" issue. One simple possibility is for each agent ℓ to randomize somehow the control choices of other agents $j \neq \ell$ when choosing its own control, particularly in "tightly coupled" cases where the choice of agent ℓ is "strongly" affected by the choices of the agents $j \neq \ell$.

An alternative idea is to choose the signaling policy $\widehat{\mu}_k$ to approximate the sequential multiagent rollout policy (the one computed with each agent knowing the controls applied by the preceding agents), or some other policy that is known to embody coordination between the agents. In particular, we may obtain $\widehat{\mu}_k$ as the multiagent rollout policy for a related but simpler problem, such as a certainty equivalent version of the original problem, whereby the stochastic system is replaced by a deterministic one.

Another interesting possibility is to compute $\widehat{\mu}_k = (\widehat{\mu}_k^1, \ldots, \widehat{\mu}_k^m)$ by off-line training of a neural network (or m networks, one per agent) with training samples generated through the sequential multiagent rollout policy. We intuitively expect that if the neural network provides a signaling policy that approximates well the sequential multiagent rollout policy, we would obtain better performance than the base policy. This expectation was confirmed in a case study involving a large-scale multi-robot repair application (see [BKB20]).

The advantage of autonomous multiagent rollout with neural network or other approximations is that it may lead to approximate policy improvement, while at the same time allowing asynchronous agent operation without coordination through communication of their rollout control values (but still assuming knowledge of the exact state by all agents).

2.10 ROLLOUT FOR BAYESIAN OPTIMIZATION AND SEQUENTIAL ESTIMATION

In this section, we discuss a wide class of problems that has been studied intensively in statistics and related fields since the 1940s. Roughly speaking, in these problems we use observations and sampling for the purpose of inference, but the number and the type of observations are not fixed in advance. Instead, the outcomes of the observations are sequentially evaluated on-line with a view towards stopping or modifying the observation process. This involves sequential decision making, thus bringing to bear exact and

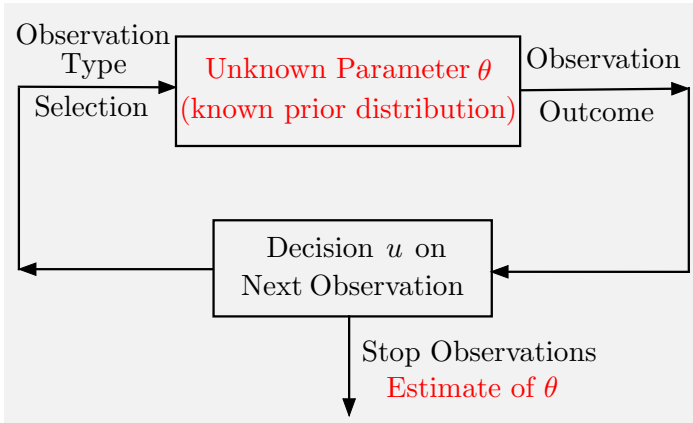

Figure 2.10.1 Illustration of sequential estimation of a parameter θ. At each time a decision is made to select one of several observation types relating to θ, each of different cost, or stop the observations and provide a final estimate of θ.

approximate DP. A central issue here is to estimate an m-dimensional random vector θ, using optimal sequential selection of observations, which are based on feedback from preceding observations; see Fig. 2.10.1. Here is a simple but historically important illustrative example, where θ represents a binary hypothesis.

Example 2.10.1 (Hypothesis Testing - Sequential Probability Ratio Test)

Consider a hypothesis testing problem whereby we can make observations, at a cost C each, relating to two hypotheses. Given a new observation, we can either accept one of the hypotheses or delay the decision for one more period, pay the cost C, and obtain a new observation. At issue is trading off the cost of observation with the higher probability of accepting the wrong hypothesis. As an example, in a quality control setting, the two hypotheses may be that a certain product meets or does not meet a certain level of quality, while the observations may consist of quantitative tests of the quality of the product.

Intuitively, one expects that once the conditional probability of one of the hypotheses, given the observations thus far, gets sufficiently close to 1, we should stop the observations. Indeed classical DP analyses bear this out; see e.g., the books by Chernoff [Che72], DeGroot [DeG70], Whittle [Whi82], and the references quoted therein. In particular, the simple version of the hypothesis testing problem just described admits a simple and elegant optimal solution, known as the *sequential probability ratio test*. On the other hand more complex versions of the problem, involving for example multiple hypotheses and/or multiple types of observations, are computationally intractable, thus necessitating the use of suboptimal approaches.

An important distinction in sequential estimation problems is whether the current choice of observation affects the cost and the availability of future observations. If this is so, the problem can often be viewed most fruitfully as a *combined estimation and control problem*, and is related to a type of *adaptive control* problem that we will discuss in the next section. As an example we will consider there sequential decoding, whereby we search for a hidden code word by using a sequence of queries, in the spirit of the Wordle puzzle and the family of Mastermind games [see, e.g., the Wikipedia page for "Mastermind (board game)"].

If the observation choices are "independent" and do not affect the cost or availability of future observations, the problem is substantially simplified. We will discuss problems of this type in the present section, starting with the cases of surrogate and Bayesian optimization.

Surrogate Optimization

Surrogate optimization refers to a collection of methods, which address suboptimally a broad range of minimization problems, beyond the realm of DP. The problem is to minimize approximately a function that is given as a "black box." By this we mean a function whose analytical expression is unknown, and whose values at any one point may be hard-to-compute, e.g., may requite costly simulation or experimentation. The idea is to replace such a cost function with a "surrogate" whose values are easier to compute.

Here we introduce a model of the cost function that is parametrized by a parameter θ; see Fig. 2.10.2. We observe sequentially the cost function at a few observation points, construct a model of the cost function (the surrogate) by estimating θ based on the results of the observations, and minimize the surrogate to obtain a suboptimal solution. The question is how to select observation points sequentially, using feedback from previous observations. This selection process often embodies an *exploration-exploitation tradeoff*: Observing at points likely to have near-optimal value vs observing at points in relatively unexplored areas of the search space.

Surrogate optimization at its core involves construction from data of functions of interest. Thus the ideas to be presented apply to other domains, e.g., the construction of probability density functions from data.

Bayesian Optimization

Bayesian optimization (BO) has been used widely for the approximate optimization of functions whose values at given points can only be obtained through time-consuming calculation, simulation, or experimentation. A classical application from geostatistical interpolation, pioneered by the statisticians Matheron and Krige, was to identify locations of high gold distribution in South Africa based on samples from a few boreholes (the name "kriging" is often used to refer to this type of application; see the review by Kleijnen [Kle09]). As another example, BO has been used to select the

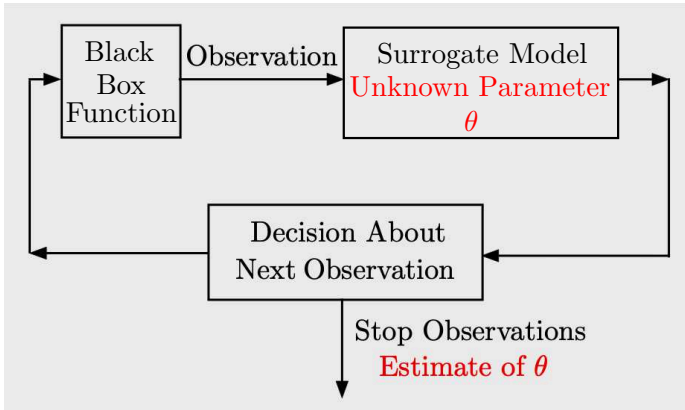

Figure 2.10.2 Illustration of the construction of a surrogate for a "black box" function f whose values are hard-to-compute. We replace f with a parametric model that involves a parameter θ to be estimated by using observations at some points. The points are selected sequentially, using the results of earlier observations. Eventually, the observation process is stopped (often when an observation/computation budget limit is reached), and the final estimate of θ is used to construct the surrogate to be minimized in place of f.

hyperparameters of machine-learning models, including the architectural parameters of the deep neural network of AlphaZero; see [SHS17].

In this section, we will focus on a relatively simple BO formulation that can be viewed as the special case of surrogate optimization. In particular, we will discuss the case where the surrogate function is parametrized by the collection of its values at the points where it is defined.† See the references cited later in this section. Formally, we want to minimize a real-valued function f, defined over a set of m points, which we denote by $1, \ldots, m$. These m points lie in some space, which we leave unspecified for the moment.‡ The values of the function are not readily available, but can be estimated with observations that may be imperfect. However, the observations are so costly that we can only hope to observe the function at a limited number of points. Once the function has been estimated with this type of observation process, we obtain a surrogate cost function, which may be minimized to obtain an approximately optimal solution.

† More complex forms of surrogates are obtained through linear combinations of some basis functions, with the parameter vector θ consisting of the weights of the basis functions.

‡ We restrict the domain of definition of f to be the finite set $\{1, \ldots, m\}$ in order to facilitate the implementation of the rollout algorithm to be discussed in what follows. However, in a more general and sometimes more convenient formulation, the domain of f can be an infinite set, such as a subset of a finite-dimensional Euclidean space.

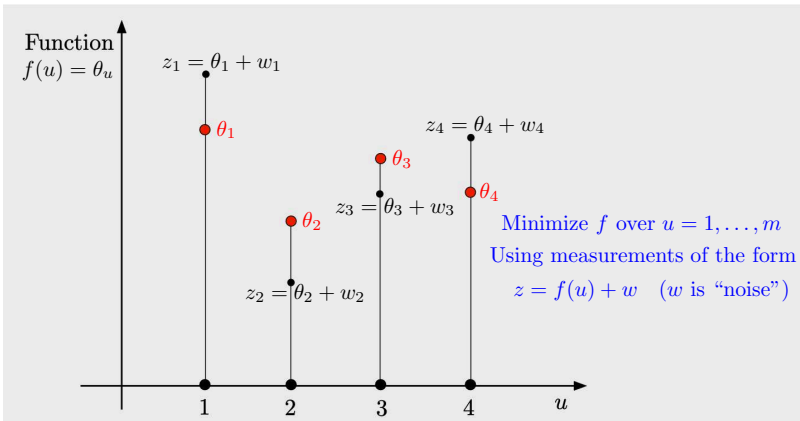

Figure 2.10.3 Illustration of a function f that we wish to estimate. The function is defined at the points $u = 1, 2, 3, 4$, and is represented by a vector $\theta = (\theta_1, \theta_2, \theta_3, \theta_4) \in \Re^4$, in the sense that $f(u) = \theta_u$ for all u. The prior distribution of θ is given, and is used to construct the posterior distribution of θ given noisy observations $z_u = \theta_u + w_u$ at some of the points u.

We denote the value of f at a point u by θ_u:

$$\theta_u = f(u), \qquad \text{for all } u = 1, \ldots, m.$$

Thus the m-dimensional vector $\theta = (\theta_1, \ldots, \theta_m)$ belongs to $\Re^m$ and represents the function f. We assume that we obtain sequentially noisy observations of values $f(u) = \theta_u$ at suitably selected points u. These values are used to estimate the vector θ (i.e., the function f), and to ultimately minimize (approximately) f over the m points $u = 1, \ldots, m$. The essence of the problem is to select points for observation based on an exploration-exploitation tradeoff (exploring the potential of relatively unexplored candidate solutions and improving the estimate of promising candidate solutions). The fundamental idea of the BO methodology is that the function value changes relatively slowly, so that observing the function value at some point provides information about the function values at neighboring points. Thus a limited number of strategically chosen observations can provide reasonable approximation to the true cost function over a large portion of the search space.

For a mathematical formulation of a BO framework, we assume that at each of N successive times $k = 1, \ldots, N$, we select a single point $u_k \in \{1, \ldots, m\}$, and observe the corresponding component θ_{u_k} of θ (i.e., the function value at u_k) with some noise w_{u_k}, i.e.,

$$z_{u_k} = \theta_{u_k} + w_{u_k}; \tag{2.93}$$

see Fig. 2.10.3. We view the observation points $u_1, \ldots, u_N$ as the optimization variables (or controls/actions in a DP/RL context), and consider policies for selecting u_k with knowledge of the preceding observations

$z_{u_1}, \ldots, z_{u_{k-1}}$ that have resulted from the selections $u_1, \ldots, u_{k-1}$. We assume that the noise random variables w_u, $u \in \{1, \ldots, m\}$ are independent and that their distributions are given. Moreover, we assume that θ has a given a priori distribution on the space of m-dimensional vectors $\Re^m$, which we denote by b_0. The posterior distribution of θ, given any subset of observations

$$\{z_{u_1}, \ldots, z_{u_k}\},$$

is denoted by b_k.

An important special case arises when b_0 and the distributions of w_u, $u \in \{1, \ldots, m\}$, are Gaussian. In this case b_0 is a multidimensional Gaussian distribution, defined by its mean (based on prior knowledge, or an equal value for all $u = 1, \ldots, m$ in case of absence of such knowledge) and its covariance matrix [implying greater correlation for pairs (u, u') that are "close" to each other in some problem-specific sense, e.g., exponentially decreasing with the Euclidean distance between u and u']. A key consequence of this assumption is that the posterior distribution b_k is multidimensional Gaussian, and can be calculated in closed form by using well-known formulas.

More generally, b_k evolves according to an equation of the form

$$b_{k+1} = B_k(b_k, u_{k+1}, z_{u_{k+1}}), \qquad k = 0, \ldots, N-1. \tag{2.94}$$

Thus given the set of observations up to time k, and the next choice u_{k+1}, resulting in an observation value $z_{u_{k+1}}$, the function B_k gives the formula for updating b_k to b_{k+1}, and may be viewed as a recursive estimator of b_k. In the Gaussian case, the function B_k can be written in closed form, using standard formulas for Gaussian random vector estimation. In other cases where no closed form expression is possible, B_k can be implemented through simulation that computes (approximately) the new posterior b_{k+1} using samples generated from the current posterior b_k.

At the end of the sequential estimation process, after the complete observation set

$$\{z_{u_1}, \ldots, z_{u_N}\}$$

has been obtained, we have the posterior distribution b_N of θ, which we can use to compute a surrogate of f. As an example we may use as surrogate the posterior mean

$$\hat{\theta} = (\hat{\theta}_1, \ldots, \hat{\theta}_m),$$

and declare as minimizer of f over u the point u^* with minimum posterior mean:

$$u^* \in \arg\min\{\hat{\theta}_u \mid u = 1, \ldots, m\};$$

see Fig. 2.10.4.

There is a large literature relating to the surrogate and Bayesian optimization methodology and its applications, particularly for the Gaussian

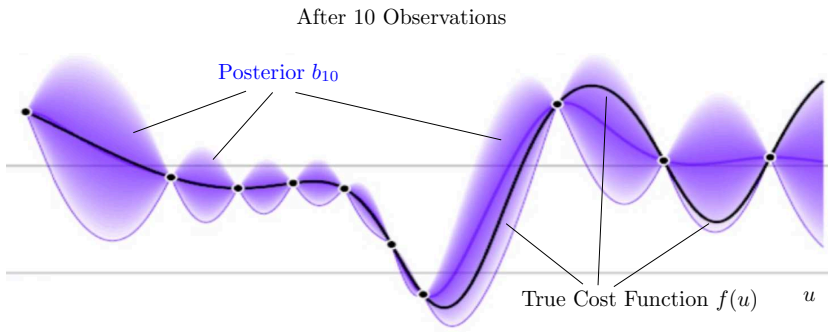

Figure 2.10.4 Illustration of the true cost function f, defined over an interval of the real line, and the posterior distribution b_{10} after noise-free measurements at 10 points. The shaded area represents the interval of the mean plus/minus the standard deviation of the posterior b_{10} at the points u. The mean of the finally obtained posterior, as a function of u, may be viewed as a surrogate cost function that can be minimized in place of f. Note that since the observations are assumed noise-free, the mean of the posterior is exact at the observation points.

case. We refer to the books by Rasmussen and Williams [RaW06], Powell and Ryzhov [PoR12], the highly cited papers by Saks et al. [SWM89], Jones, Schonlau, and Welch [JSW98], and Queipo et al. [QHS05], the reviews by Sasena [Sas02], Powell and Frazier [PoF08], Forrester and Keane [FoK09], Kleijnen [Kle09], Brochu, Cora, and De Freitas [BCD10], Ryzhov, Powell, and Frazier [RPF12], Ghavamzadeh, Mannor, Pineau, and Tamar [GMP15], Shahriari et al. [SSW16], and Frazier [Fra18], and the references quoted there. Our purpose here is to focus on the aspects of the subject that are most closely connected to exact and approximate DP.

A Dynamic Programming Formulation

The sequential estimation problem just described, viewed as a DP problem, involves a state at time k, which is the posterior (or belief state) b_k, and a control/action at time k, which is the point index u_{k+1} selected for observation. The transition equation according to which the state evolves, is

$$b_{k+1} = B_k(b_k, u_{k+1}, z_{u_{k+1}}), \qquad k = 0, \ldots, N-1;$$

cf. Eq. (2.94). To complete the DP formulation, we need to introduce a cost structure. To this end, we assume that observing θ_u, as per Eq. (2.93), incurs a cost $c(u)$, and that there is a terminal cost $G(b_N)$ that depends of the final posterior distribution; as an example, the function G may involve the mean and covariance corresponding to b_N.

The corresponding DP algorithm is given by

$$J_k^*(b_k) = \min_{u_{k+1}\in\{1,\ldots,m\}} \Bigg[c(u_{k+1}) + E_{z_{u_{k+1}}} \Big\{ J_{k+1}^*\big(B_k(b_k, u_{k+1}, z_{u_{k+1}})\big) \mid b_k, u_{k+1} \Big\} \Bigg], \tag{2.95}$$

and proceeds backwards from the terminal condition

$$J_N^*(b_N) = G(b_N). \tag{2.96}$$

The expected value in the right side of the DP equation (2.95) is taken with respect to the conditional distribution of $z_{u_{k+1}}$, given b_k and the choice u_{k+1}. The observation cost $c(u)$ may be 0 or a constant for all u, but it can also have a more complicated dependence on u. The terminal cost $G(b_N)$ may be a suitable measure of surrogate "fidelity" that depends on the posterior mean and covariance of θ corresponding to b_N.

Generally, executing the DP algorithm (2.95) is practically infeasible, because the space of posterior distributions is infinite-dimensional. In the Gaussian case where the a priori distribution b_0 is Gaussian and the noise variables w_u are Gaussian, the posterior b_k is m-dimensional Gaussian, so it is characterized by its mean and covariance, and can be specified by a finite set of numbers. Despite this simplification, the DP algorithm (2.95) is prohibitively time-consuming even under Gaussian assumptions, except for simple special cases. We consequently resort to approximation in value space, whereby the function J_{k+1}^* in the right side of Eq. (2.95) is replaced by an approximation $\tilde{J}_{k+1}$.

Approximation in Value Space

The most popular BO methodology makes use of a myopic/greedy policy μ_{k+1}, which at each time k and given b_k, selects a point $\hat{u}_{k+1} = \mu_{k+1}(b_k)$ for the next observation, using some calculation involving an *acquisition function*. This function, denoted $A_k(b_k, u_{k+1})$, quantifies some form of "expected benefit" for an observation at u_{k+1}, given the current posterior b_k.† The myopic policy selects the next point at which to observe, $\hat{u}_{k+1}$,

† A common type of acquisition function is the *upper confidence bound*, which has the form

$$A_k(b_k, u) = T_k(b_k, u) + \beta R_k(b_k, u),$$

where $T_k(b_k, u)$ is the negative of the mean of $f(u)$ under the posterior distribution b_k, $R_k(b_k, u)$ is the standard deviation of $f(u)$ under the posterior distribution b_k, and β is a tunable positive scalar parameter. Thus $T_k(b_k, u)$ can be

by maximizing the acquisition function:

$$\hat{u}_{k+1} \in \arg \max_{u_{k+1} \in \{1,\ldots,m\}} A_k(b_k, u_{k+1}). \tag{2.97}$$

Several ways to define suitable acquisition functions have been proposed, and an important issue is to be able to calculate economically its values $A_k(b_k, u_{k+1})$ for the purposes of the maximization in Eq. (2.97). Another important issue of course is to be able to calculate the posterior b_k economically.

Approximation in value space is an alternative approach, which is based on the DP formulation of the preceding section. In particular, in this approach we approximate the DP algorithm (2.95) by replacing J^*_{k+1} with an approximation $\tilde{J}_{k+1}$ in the minimization of the right side. Thus we select the next observation at point $\tilde{u}_{k+1}$ according to

$$\tilde{u}_{k+1} \in \arg \min_{u_{k+1} \in \{1,\ldots,m\}} Q_k(b_k, u_{k+1}), \qquad k = 0, \ldots, N-1, \tag{2.98}$$

where $Q_k(b_k, u_{k+1})$ is the Q-factor corresponding to the pair (b_k, u_{k+1}), given by

$$Q_k(b_k, u_{k+1}) = c(u_{k+1}) + E_{z_{u_{k+1}}} \Big\{ \tilde{J}_{k+1}\big(B_k(b_k, u_{k+1}, z_{u_{k+1}})\big) \mid b_k, u_{k+1} \Big\}. \tag{2.99}$$

The expected value in the preceding equation is taken with respect to the conditional probability distribution of $z_{u_{k+1}}$ given (b_k, u_{k+1}), which can be computed using b_k and the given distribution of the noise $w_{u_{k+1}}$. Thus if b_k and $\tilde{J}_{k+1}$ are available, we may use Monte Carlo simulation to determine the Q-factors $Q_k(b_k, u_{k+1})$ for all $u_{k+1} \in \{1, \ldots, m\}$, and select as next point for observation the one that corresponds to the minimal Q-factor [cf. Eq. (2.98)].

Rollout Algorithms for Bayesian Optimization

A special case of approximation in value space is the rollout algorithm, whereby the function J^*_{k+1} in the right side of the DP Eq. (2.95) is replaced by the cost function of some base policy $\mu_{k+1}(b_k)$, $k = 0, \ldots, N-1$. Thus,

---

viewed as an *exploitation index* (encoding our desire to search within parts of the space where f takes low value), while $R_k(b_k, u)$ can be viewed as an *exploration index* (encoding our desire to search within parts of the space that are relatively unexplored). There are several other popular acquisition functions, which directly or indirectly embody a tradeoff between exploitation and exploration. A popular example is the *expected improvement* acquisition function, which is equal to the expected value of the reduction of $f(u)$ relative to the minimal value of f obtained up to time k (under the posterior distribution b_k).

given a base policy the rollout algorithm uses the cost function of this policy as the function $\tilde{J}_{k+1}$ in the approximation in value space scheme (2.98)-(2.99). The values of $\tilde{J}_{k+1}$ needed for the Q-factor calculations in Eq. (2.99) can be computed or approximated by simulation. Greedy/myopic policies based on an acquisition function [cf. Eq. (2.97)] have been suggested as base policies in various rollout proposals.†

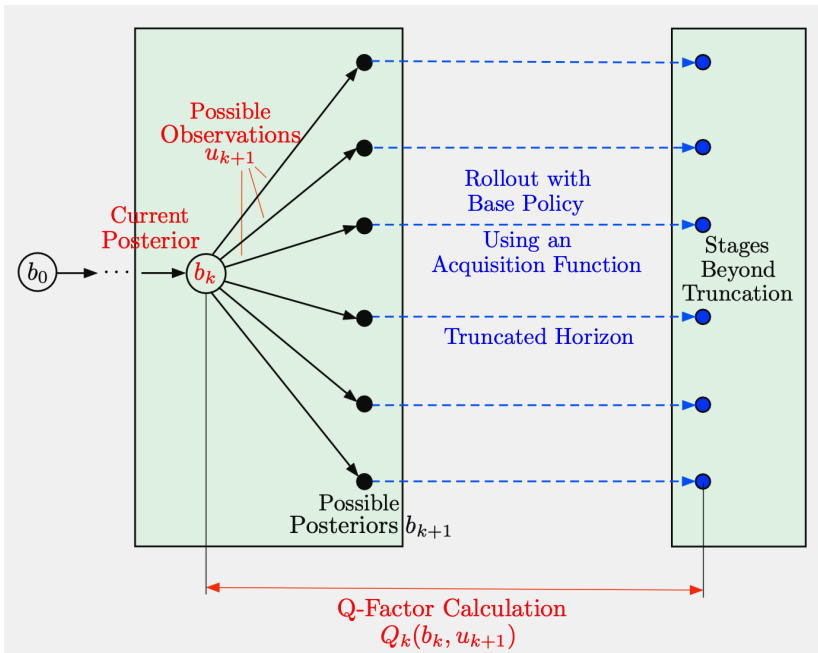

Figure 2.10.5 Illustration of rollout at the current posterior b_k. For each $u_{k+1} \in \{1, \ldots, m\}$, we compute the Q-factor $Q_k(b_k, u_{k+1})$ by using Monte-Carlo simulation with samples from $w_{u_{k+1}}$ and a base heuristic that uses an acquisition function starting from each possible posterior b_{k+1}. The rollout may extend to the end of the horizon N, or it may be truncated after a few steps.

In particular, given b_k, the rollout algorithm computes for each $u_{k+1} \in$

† The rollout algorithm for BO was first proposed under Gaussian assumptions by Lam, Wilcox, and Wolpert [LWW16]. It was further discussed by Jiang et al. [JJB20], [JCG20], Lee at al. [LEC20], Lee [Lee20], Yue and Kontar [YuK20], Lee et al. [LEP21], Paulson, Sorouifar, and Chakrabarty [PSC22], where it is also referred to as "nonmyopic BO" or "nonmyopic sequential experimental design." For related work, see Gerlach, Hoffmann, and Charlish [GHC21]. These papers also discuss various approximations to the rollout approach, and generally report encouraging computational results. Section 3.5 of the author's book [Ber20a] focuses on rollout algorithms for surrogate and Bayesian optimization.